

Prova Finale - Progetto di Reti Logiche

Angelo Rubino - 10858548 - 211166

Mirco Poma - 10839115 - 211383

Istituto: Politecnico di Milano**Materia:** Reti Logiche [2024/2025]**Docente:** Prof. Palermo Gianluca

Abstract	2
Introduzione	2
Interfacce	2
Funzionamento	3
Esempio:	5
Memoria RAM	6
Architettura	7
Macchina a stati	7
Segnali introdotti nel Modulo progettato	7
Spiegazione degli stati	8
STATO S0	8
STATO S1	8
STATO S2	8
STATO S3	8
STATO S4	9
STATO S5	9
STATO S6	9
STATO S7	9
STATO S8	9
STATO S9	10
STATO DONE	10
Processi utilizzati	10
Scelte progettuali per il processo “filtro”	10
Gestione dei Segnale o_mem_add	11
Risultati Sperimentali	12
Report di Sintesi	12
Test bench (a)	12
Test bench (b)	12
Test bench (c)	13
Test bench (d)	13
Conclusioni	14

Abstract

La presente relazione descrive la nostra personale soluzione alla specifica assegnata. Astraendo, il componente attende il segnale di “start” per iniziare la lettura dei dati a partire dall’indirizzo fornito in ingresso. Tra di essi si trovano il numero di valori da elaborare, quale filtro applicarvi e i coefficienti da usare per ciascun filtro. Subito dopo le istruzioni, nella memoria sono contenuti i numeri su cui eseguire le operazioni: il dispositivo ne legge 7 alla volta per poi eseguire le moltiplicazioni, la normalizzazione e infine il risultato viene salvato in memoria. Se sono presenti altri valori, si torna allo stato di lettura, altrimenti viene alzato il segnale di “done” per indicare il termine dell’elaborazione.

Introduzione

Interfacce

Segnale	Descrizione	Numero di bit	Tipo del segnale
<i>i_clk</i>	Segnale di Clock, unico per il sistema.	1	Ingresso
<i>i_rst</i>	Segnale di Reset Asincrono, unico per il sistema.	1	Ingresso
<i>i_start</i>	Segnale che indica che è possibile partire con l’elaborazione dei dati se alzato.	1	Ingresso
<i>i_add</i>	Indirizzo dal quale iniziano i dati.	16	Ingresso
<i>i_mem_data</i>	Dato che si sta leggendo dalla memoria.	8	Ingresso
<i>o_done</i>	Segnale che indica la terminazione dell’elaborazione se alzato.	1	Uscita
<i>o_mem_addr</i>	Dato che si vuole scrivere in memoria.	16	Uscita
<i>o_mem_data</i>	Indirizzo della memoria nel quale si vuole scrivere/leggere un valore.	8	Uscita
<i>o_mem_we</i>	Segnale che abilita la scrittura in memoria quando alzato.	1	Uscita
<i>o_mem_en</i>	Segnale che abilita la memoria quando alzato.	1	Uscita

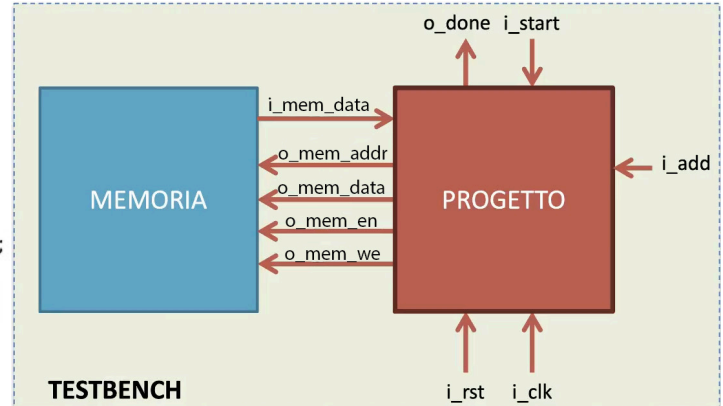
```

entity project_reti_logiche is
  port (
    i_clk   : in std_logic;
    i_rst   : in std_logic;
    i_start : in std_logic;
    i_add   : in std_logic_vector(15 downto 0);

    o_done : out std_logic;

    o_mem_addr : out std_logic_vector(15 downto 0);
    i_mem_data : in std_logic_vector(7 downto 0);
    o_mem_data : out std_logic_vector(7 downto 0);
    o_mem_we   : out std_logic;
    o_mem_en   : out std_logic
  );
end project_reti_logiche;

```

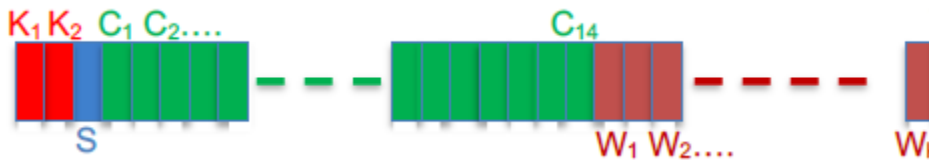


Funzionamento

All'istante iniziale, quello relativo al reset del sistema, le uscite hanno i seguenti valori:

<i>o_done</i>	0
<i>o_mem_addr</i>	DC
<i>o_mem_data</i>	DC
<i>o_mem_we</i>	0
<i>o_mem_en</i>	0

I dati in ingresso sono letti sul fronte di salita del clock e sono organizzati nel seguente modo all'interno della memoria:



- K_1, K_2 : due Byte che indicano il numero di valori da leggere dalla memoria, da qui in poi indicato come K .
- S : un Byte che indica quale filtro usare a seconda del bit meno significativo (0 per il filtro di ordine 3 e 1 per il filtro di ordine 5).
- $C_1 - C_{14}$: 14 Byte per i valori da usare nei filtri di ordine 3 ($C_1 - C_7$) e di ordine 5 ($C_8 - C_{14}$).

- $W_1 - W_k$: K valori su cui applicare il filtro.

Prima dell'elaborazione del primo dato, il modulo attende il segnale di reset dato da i_rst ; quando questo torna a 0 il modulo inizia a funzionare solo dopo che viene alzato il segnale i_start .

A questo punto il componente legge i 17 Byte di preambolo e successivamente i dati nella memoria a cui applica il filtro differenziale con una delle configurazioni date, indicata dal terzo Byte letto dalla configurazione.

I possibili filtri sono di ordine 3 e di ordine 5. Essi prevedono una sommatoria di ogni dato in ingresso, moltiplicato per il corrispettivo coefficiente del filtro secondo la formula $\sum C_j * f[j+i]$ dove $j \in [-2, +2]$ per il filtro di ordine 3 e $j \in [-3, +3]$ per quello di ordine 5. Infine, si divide questo risultato intermedio per un fattore di normalizzazione (12 nel caso del filtro di ordine 3, 60 nel caso di filtro di ordine 5); tale operazione viene effettuata sfruttando degli shift a destra, ognuno dei quali rappresenta una divisione per due. $1/12$ è approssimato come $1/16 + 1/64 + 1/256 + 1/1024$ mentre $1/60$ è approssimato come $1/64 + 1/1024$.

Si noti che per numeri negativi, ogni shift a destra produce un troncamento verso -1. Per ridurre questo errore si aggiunge +1 al risultato del singolo shift, quando il valore fornito è negativo.

Dopodiché, il componente scrive ogni valore filtrato nella memoria a partire dall'indirizzo $i_add + 17 + K$, ossia il primo indirizzo disponibile successivo ai valori di ingresso.

Il segnale i_start rimane alto fino a che o_done non è portato ad 1 e ciò avviene solo quando tutti i W_k dati sono stati elaborati e scritti in memoria. o_done rimane poi alto finché i_start non è portato nuovamente a 0.

Dal momento in cui il segnale o_done viene abbassato, un nuovo segnale i_start può essere alzato per indicare così l'inizio di una nuova elaborazione.

Il modulo è progettato considerando che prima del primo $i_start = 1$ è sempre dato il segnale di reset da i_rst ; prima del primo segnale di reset il comportamento del modulo non è specificato. Inoltre, i_rst può anche essere alzato durante un'elaborazione (quando $i_start = 1$ e $o_done = 0$): in questo caso si torna ad attendere che venga alzato il segnale i_start .

Esempio:

Funzione da applicare: $f'(i) = (1/n) * \sum C_j * f[j+i]$

$j \in [-2, +2]$ per il filtro di ordine 3 e $j \in [-3, +3]$ per quello di ordine 5.

$n=12$ per il filtro di ordine 3 e $n=60$ per quello di ordine 5.

Coefficienti del filtro di ordine 3:

[0, -1, 8, 0, -8, 1, 0]

i dati **in grassetto** sono sempre da considerare ghost zero nel filtro di ordine 3.

Sequenza di partenza ($W_1 - W_k$):

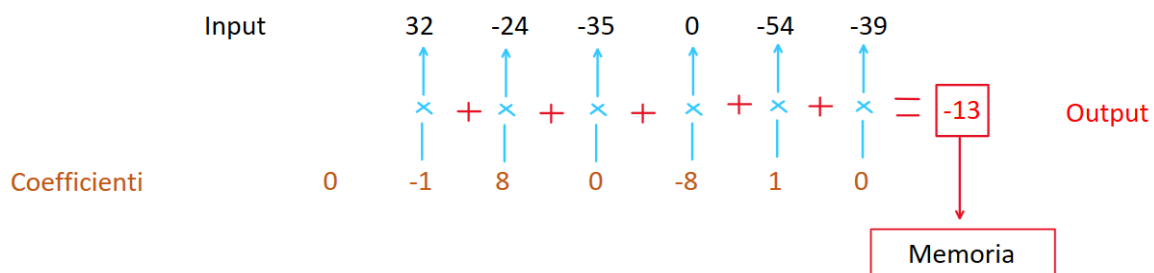
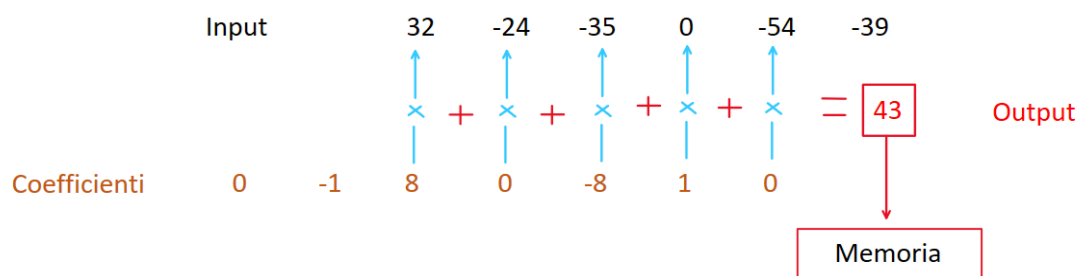
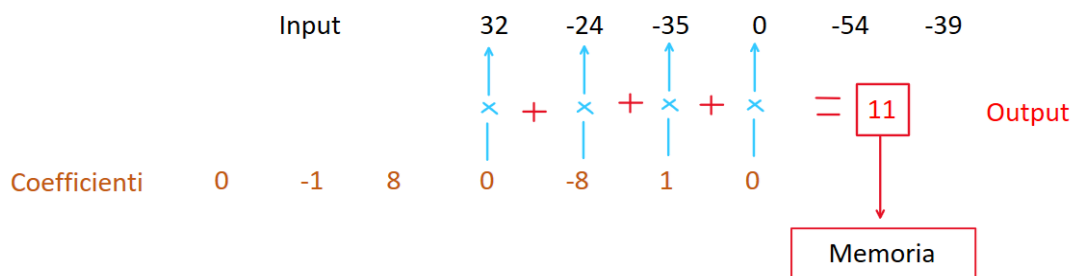
32 -24 -35 0 46 -54 -39 -22 -53 -47 12 11 11 45 -30

Dopo l'applicazione del filtro di ordine 3:

157 536 -178 -678 428 658 -355 119 251 -487 -400 100 -314 303 426

A seguito della normalizzazione (sequenza finale $R_1 - R_k$):

11 43 -13 -54 33 53 -28 8 18 -38 -31 7 -24 23 33



Memoria RAM

Essa è fornita e non è da sintetizzare. La memoria in questione è la seguente:

```
-- Single-Port Block RAM Write-First Mode (recommended template)
--
-- File: rams_02.vhd
--
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
entity rams_sp_wf is
port(
    clk  : in  std_logic;
    we   : in  std_logic;
    en   : in  std_logic;
    addr : in  std_logic_vector(15 downto 0);
    di   : in  std_logic_vector(7 downto 0);
    do   : out std_logic_vector(7 downto 0)
);
end rams_sp_wf;

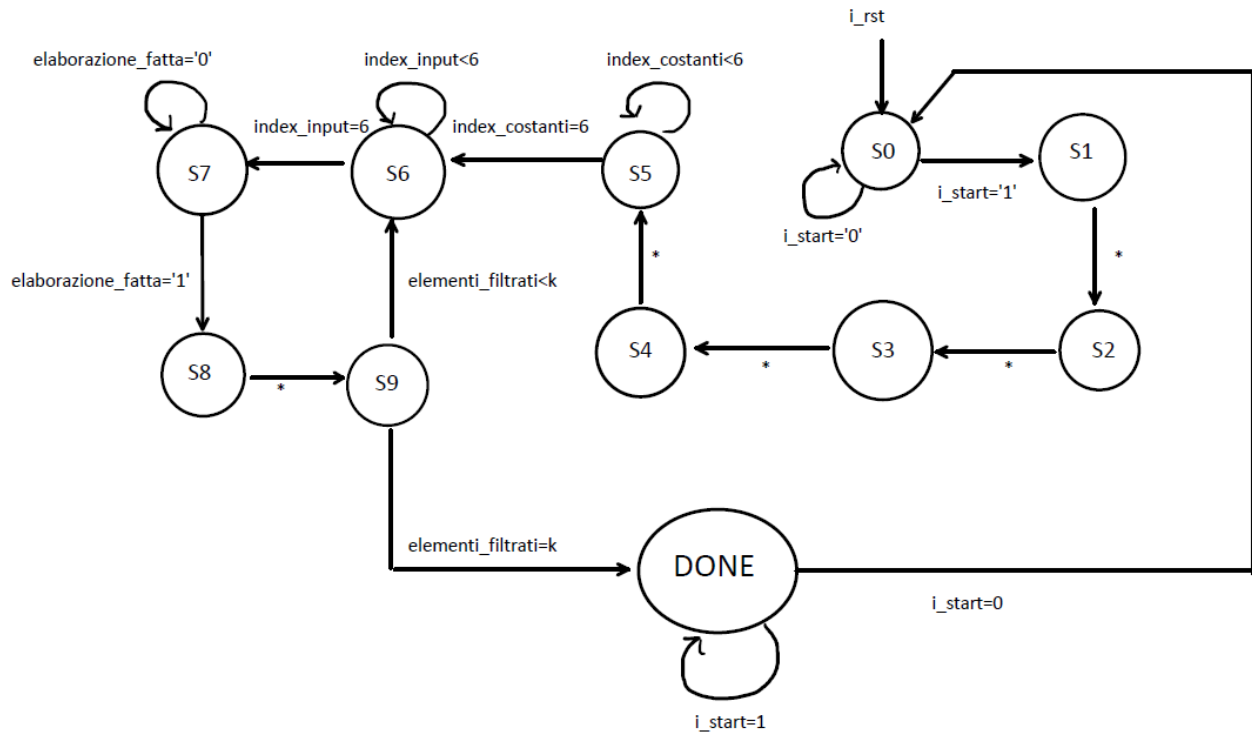
architecture syn of rams_sp_wf is
type ram_type is array (65535 downto 0) of std_logic_vector(7 downto 0);
signal RAM : ram_type;
begin
    process(clk)
    begin
        if clk'event and clk = '1' then
            if en = '1' then
                if we = '1' then
                    RAM(conv_integer(addr)) <= di;
                    do <= di after 2 ns;
                else
                    do <= RAM(conv_integer(addr)) after 2 ns;
                end if;
            end if;
        end if;
    end process;
end syn;
```

Architettura

L'architettura è costituita da un singolo modulo che legge gli ingressi, li memorizza, applica il filtro e infine li salva nell'indirizzo subito successivo alla sequenza di input.

Il software utilizzato è stato Xilinx Vivado (versione 2024.2); L'FPGA target è la Xilinx Artix-7 xc7a50tfgg484-3.

Macchina a stati



Segnali introdotti nel Modulo progettato

Nome	Tipo	Scopo	Numero di bit
<i>k</i>	std_logic_vector	Salva negli 8 bit più significativi K1 e K2 negli 8 bit meno significativi.	16
<i>elementi_filtrati</i>	std_logic_vector	Tiene il conto degli elementi filtrati e scritti in memoria.	16
<i>tipo_filtro</i>	std_logic	Salva il tipo di filtro da usare.	1
<i>elaborazione_fatta</i>	std_logic	Segnala che è appena avvenuto il filtraggio di uno dei dati di input.	1

<i>memory_address</i>	std_logic_vector	Salva l'indirizzo che a seconda dello stato sarà poi assegnato ad <i>o_mem_addr</i> e quindi letto dalla memoria.	16
<i>input_start</i>	std_logic_vector	Mantiene inizialmente l'indirizzo di memoria del secondo dato da leggere e poi tutti i successivi dati da leggere (Non include i 17 byte di preambolo).	16
<i>write_address</i>	std_logic_vector	Mantiene il primo indirizzo libero in cui scrivere i valori di output e poi tutti i successivi.	16
<i>costanti</i>	array	Salva le costanti per il filtro selezionato.	7 x 8 bit
<i>input</i>	array	Salva temporaneamente parte della sequenza dei K dati di input (Non i 17 bit di preambolo)	7 x 8 bit
<i>index_costanti</i>	integer	Indice per scorrere il vettore delle costanti.	3
<i>index_input</i>	integer	Indice per scorrere il vettore dell'input.	3

Spiegazione degli stati

STATO S0

È lo stato di RESET: si itera su S0 fintanto che l'ingresso *i_start* è 0 mantenendo *o_mem_en* basso e *o_mem_addr* a DC finché *i_start* non viene alzato. Dopodiché inizia la lettura del preambolo negli stadi successivi.

STATO S1

Si legge K_1 e lo si memorizza negli otto bit più significativi di k .

STATO S2

Si legge K_2 e lo si memorizza negli otto bit meno significativi di k

STATO S3

Si legge S , il Byte che identifica il tipo di filtro; successivamente si memorizza su *write_address* il primo indirizzo valido per la prima scrittura, operazione eseguita in questo stato perché il

valore di K è già disponibile per il calcolo. A seconda del tipo di filtro riconosciuto, si salva sul segnale *memory_address* l'indirizzo dal quale iniziare a leggere i coefficienti del filtro.

STATO S4

Il suo scopo è aggiornare la memoria e passare *memory_address*, individuato nello stato precedente, al segnale di uscita *o_mem_add* in modo che la memoria possa spostarsi sulla cella a tale indirizzo nello stato successivo.

STATO S5

Viene fatta la lettura delle costanti: per ognuna si incrementa di 1_{10} il valore di *index_costanti* e *memory_address*. Definiamo 'i' come l'indirizzo del primo dato di input da leggere. Alla lettura della penultima costante, se il filtro precedentemente identificato è quello di ordine 3, in *memory_address* viene salvato l'indirizzo (i-2); altrimenti, se il filtro è quello di ordine 5, viene memorizzato (i-3). In modo del tutto analogo su *input_start* viene salvato l'indirizzo del secondo dato da leggere. Queste operazioni (i-2, i-3) vengono fatte per gestire i ghost zero da usare in S6. Quando il valore di *index_costanti* è maggiore o uguale a 6, si assume che la sequenza di costanti sia finita e si passa allo stato S6.

STATO S6

Vengono letti 7 valori di input in modo da prendere abbastanza dati per poter applicare entrambi i filtri, incrementando ad ogni lettura il valore di *index_input* e *memory_address* di 1_{10} . Questo stato, inoltre, gestisce i ghost zero all'inizio e alla fine del vettore input: sfruttando lo spostamento di posizioni svolto nello stato precedente (i-2, i-3), se si rileva un indirizzo che è fuori da quello degli input effettivi, ossia dall'indirizzo "i" in poi, viene inserito uno 0 nel vettore *input*.

STATO S7

index_input viene riportato a 0_{10} in modo da predisporre la macchina per la lettura successiva e *memory_address* viene portato al valore di *input_start*. Successivamente, si esegue il filtraggio gestendo la divisione ed eventualmente il troncamento del valore, dal momento che il risultato è rappresentato su un byte. In *o_mem_address* viene messo l'indirizzo salvato su *write_address* e infine si alza il segnale *elaborazione_fatta* che permette il passaggio allo stato successivo.

STATO S8

Qui avviene la scrittura del risultato calcolato in S7: in questo stato *o_mem_we* è quindi alto. Viene inoltre incrementato di 1_{10} sia il valore di *elementi_filtrati*, inizializzato a 0_{10} nello stato di reset S0, che gli indirizzi salvati su *input_start* e *write_address*, in modo da prepararli per le operazioni di lettura e scrittura successive.

STATO S9

Questo stato viene usato per rilevare la fine dell'elaborazione: se *elementi_filtrati* è pari a *k*, si passa allo stato DONE; altrimenti si imposta *o_mem_add* all'indirizzo opportuno per continuare a leggere la sequenza di input.

STATO DONE

Viene usato per segnalare la fine dell'elaborazione, portando *o_done* ad alto.

Processi utilizzati

L'architettura utilizzata per descrivere il componente fa uso di quattro processi:

- **State_reg**: per la gestione del reset asincrono e del passaggio di stato che avviene sul fronte di salita del clock.
- **funzione_stato_prossimo**: processo asincrono che determina lo stato prossimo della FSM in base allo stato corrente e agli ingressi. Si noti che non tutti i segnali letti al suo interno compaiono nella lista di sensitività: questo perché si è cercato di mantenere il numero di segnali che potessero attivare il processo il più piccolo possibile, in modo da facilitare le operazioni di debug e manutenzione.
- **gestione_stati**: processo sincrono usato per effettuare le letture o scritture dei dati, a seconda dello stato corrente.
- **filtro**: processo sincrono il cui compito è gestire le operazioni inerenti al filtro da applicare ai dati. Utilizza sette variabili da 16 bit (*tmp1*, *tmp2*, ..., *tmp7*) per memorizzare i risultati delle moltiplicazioni tra coefficienti e valori di input, oltre che due variabili da 18 bit (*da_shiftare*, *shiftato*) per gestire risultato intermedio e la divisione tramite shift.

Scelte progettuali per il processo “filtro”

I segnali *tmp1*, *tmp2*, ..., *tmp7* sono a 16 bit, dovendo contenere il risultato del prodotto di due numeri rappresentati su 8 bit, mentre i segnali *da_shiftare* e *shiftato* sono a 18 bit dato che questo è il numero minimo di bit per coprire tutti i casi possibili per il risultato intermedio. Nel caso peggiore si può infatti avere un filtro di ordine 5 del tipo:

$[-128, -128, -128, -128, -128, -128, -128]$

Insieme al seguente input: $[-128, -128, -128, -128, -128, -128, -128]$.

Applicando il filtro sul **valore centrale** della sequenza il risultato, contenuto nel segnale *da_shiftare*, sarà pari a **114688**: numero che per essere rappresentato in complemento a 2 necessita un minimo di 18 bit.

Analogamente è possibile verificare che il numero di bit individuato è sufficiente anche nel caso pessimo per il valore intermedio negativo che risulta essere **-113792**.

Gestione dei Segnale *o_mem_add*

o_mem_add, il segnale che viene letto dalla memoria contenente la cella su cui posizionarsi per eseguire letture e scritture, è gestito tramite i segnali *memory_address*, *write_address* e *input_start*.

In *memory_address* si salvano gli indirizzi significativi utili per la lettura sia dei dati di input che dei 17 byte di preambolo, i quali vengono poi passati a *o_mem_add*.

In *write_address*, nello stato S3, viene salvato l'indirizzo della prima cella libera in cui scrivere l'output. Ogni volta che si raggiunge S8, *write_address* viene aggiornato con l'indirizzo della cella successiva a quella appena occupata. Durante la transizione da S7 allo stato successivo (S7 o S8, in base al valore di *elaborazione_fatta*), *write_address* è passato a *o_mem_addr* in modo che la memoria possa selezionare la cella in cui eseguire la scrittura.

Su *input_start* è salvato di volta in volta l'indirizzo della cella che contiene il primo numero da inserire nel vettore input; viene effettivamente usato a partire dalla seconda lettura dell'input, ossia quella che serve a generare il secondo risultato. Il suo scopo è quello di essere il punto di inizio per le letture successive alla prima. In S5 assume il suo primo valore: l'indirizzo di partenza per la seconda lettura dell'input, per poi venire aggiornato in S8 mentre viene eseguita la scrittura del dato. Ogni volta che la macchina si trova nello stato S7, viene passato a *memory_address*; quest'ultimo è a sua volta memorizzato da *o_mem_addr* quando dallo stato S9 si torna in S6.

Risultati Sperimentali

Il componente è stato controllato tramite opportuni test bench, superando tutte le verifiche a cui è stato sottoposto, sia in simulazione comportamentale che post-sintesi. Nella fase di test non sono stati presi in considerazione i ritardi dovuti alla propagazione dei segnali.

Report di Sintesi

Il componente realizzato risulta essere correttamente sintetizzabile. In particolare, per quanto riguarda la FPGA target, dal report di sintesi risulta che:

- Le **LUT** utilizzate sono 1199 cioè il 3.68% di quelle disponibili.
- Il numero di **Flip-Flop** utilizzati ammonta a 212 cioè il 0.33% di quelli disponibili.
- Il numero di **pin I/O** effettivamente utilizzati ammonta a 54 cioè il 21.6% di quelli disponibili.

Test bench (a)

Il Testbench in esame si limita a verificare semplici aspetti del componente come:

- La corretta gestione del segnale di *o_done*.
- Che l'inizio dell'elaborazione avvenga quando viene alzato il segnale *i_start*.
- La corretta elaborazione solo per il filtro di ordine 3.

Esso prevede una sola elaborazione e nessun reset escluso quello iniziale.

Il modulo risponde correttamente in questo semplice caso sia in simulazione pre-sintesi (Behavioral Simulation) sia in post-sintesi (Post-Synthesis Functional Simulation).

Test bench (b)

Il Testbench in esame verifica i seguenti aspetti del componente:

- Corretto troncamento del risultato nei casi si superino i valori 127 e -128.
- Corretto funzionamento del componente anche nel caso riceva un segnale di reset durante un'elaborazione: il reset deve funzionare correttamente e non avere effetti indesiderati sulle elaborazioni successive.
- Vengono testati entrambi i filtri.
- Verifica che il filtro di ordine 3 funzioni correttamente anche quando il primo ed ultimo coefficiente non sono zero. (Viene cioè verificato che la formula della sommatoria venga rispettata correttamente quando $j \in [-2, +2]$).
- Verifica che il componente, dopo aver terminato un'elaborazione, possa iniziarne correttamente una nuova consecutiva alla precedente.

Il modulo risponde correttamente in tutte le situazioni verificate dal Test bench sia in simulazione pre-sintesi (Behavioral Simulation) sia in post-sintesi (Post-Synthesis Functional Simulation).

Test bench (c)

Il Testbench in esame è una semplice modifica del primo. Si è cercato di riprodurre e testare la condizione descritta nella sezione precedente, ossia al paragrafo “*Scelte progettuali per il processo “filtro”*”, il caso pessimo di valori positivi, cioè quello in cui il valore intermedio del filtro sia il **maggiore possibile**.

Il modulo risponde correttamente in questo caso sia in simulazione pre-sintesi (Behavioral Simulation) sia in post-sintesi (Post-Synthesis Functional Simulation).

Test bench (d)

Come Test bench (c) ma nel caso in cui il risultato intermedio del filtro sia il **minore possibile**.

Il modulo risponde correttamente in questo caso sia in simulazione pre-sintesi (Behavioral Simulation) sia in post-sintesi (Post-Synthesis Functional Simulation).

Test bench (e)

Il Test Bench verifica il corretto funzionamento dell'intera dinamica della memoria per il filtro di ordine 5.

Il modulo risponde correttamente in questo caso sia in simulazione pre-sintesi (Behavioral Simulation) sia in post-sintesi (Post-Synthesis Functional Simulation).

Test bench (f)

Il Test Bench, come nel caso precedente, verifica il corretto funzionamento dell'intera dinamica della memoria, ma per il filtro di ordine 3.

Il modulo risponde correttamente in questo caso sia in simulazione pre-sintesi (Behavioral Simulation) sia in post-sintesi (Post-Synthesis Functional Simulation).

Test bench (g)

Il Test Bench esegue 6 elaborazioni in cascata, oltre che a presentare diversi reset. Inoltre, testa il filtro di terzo e quinto ordine e il troncamento, usando circa 48 mila celle della RAM disponibile.

Il modulo risponde correttamente in questo caso sia in simulazione pre-sintesi (Behavioral Simulation) sia in post-sintesi (Post-Synthesis Functional Simulation).

Conclusioni

È possibile ottimizzare il modulo utilizzando meno stati per la FSM di quelli effettivamente impiegati. In particolar modo si potrebbero eliminare gli stati di “setup” per la memoria (S4 ed S9) spostando le loro funzioni negli stati precedenti. Tuttavia si è scelto di non optare per questa scelta in modo da avere un codice più pulito, leggibile e manutenibile.